Observability

Observability

- The ability to understand a running system's internal state from its external outputs
  - Monitoring = watching dashboards for problems you knew might happen
  - Observability = answering questions you didn't anticipate when something unexpected breaks
- Monitoring detects known unknowns
- Observability lets you investigate unknown unknowns
- You instrument the code once
- Then ask new questions of it for the lifetime of the system

Observability

- Observability is not monitoring
- A monitored system has dashboards for the metrics that were decided in advance were important
- An observable system emits enough information that engineers can investigate failures they couldn't have predicted
  - The classic example
    - *Tour service was running fine for six months, then yesterday at 3am something happened that crashed it.*
    - *With monitoring alone, you see "service down"; then what?*
    - *With observability, you can ask "show me every request to this service in the five minutes before the crash, grouped by endpoint" and discover that one customer started sending malformed requests.*
  - The investment is the same upfront work either way; the payoff difference shows up the first time you have an incident you didn't anticipate.

The Three Pillars

- Logs
  - Discrete events that happened, with context (who, what, when)
- Metrics
  - Numeric measurements aggregated over time (counts, rates, durations)
- Traces
  - The path a single request took through one or more services
- Each pillar answers different questions; you need all three for a complete picture
- All three should be correlatable
  - A trace ID in a log connects "this user saw an error" to "here is the request that caused it"

Distributed Systems

- In a monolith
  - Attach a debugger, set a breakpoint, step through
- In a microservice mesh
  - A single user request crosses 5-15 services
  - You cannot attach a debugger to 15 services running on 15 hosts
  - "User clicks button, gets a 500 error"; where do you even start looking?
- Without observability, distributed systems are effectively undebuggable in production

Distributed Systems

- Bad log line:
  - 2026-05-29 14:23:11 INFO user 12345 logged in from 192.168.1.1
- Good log line (on the right)
  - The event name (user.login.success) is the searchable handle
  - The trace ID connects this log to the full request trace
  - Common Java tooling: SLF4J + Logback with logstash-encoder
  - Common Python tooling: the standard logging module with python-json-logger


```
{
    “timestamp": "2026-05-29T14:23:11.847Z",
    "level": "INFO",
    "service": "auth-api",
    "event": "user.login.success",
    "user_id": "12345",
    "source_ip": "192.168.1.1",
    "trace_id": "a3f7b2c1-9d4e-4b8a-8f12-3e6c7d9a1b5e"
}
```

Log Levels

- ERROR - something failed; a human needs to look at it
- WARN - something unusual but the system handled it
- INFO - normal operations worth recording (startup, shutdown, key transactions)
- DEBUG - detail for troubleshooting; usually off in production
- TRACE - extreme detail; almost never on outside local development

Log AntiPatterns

- Logging every thing at INFO level: creates noise that drowns out real signal
- Using ERROR for things that aren't actually errors, which creates pager fatigue
- Using DEBUG for things that should be INFO, then turning DEBUG on in production and overwhelming the log infrastructure
- The rule of thumb
  - ERROR should mean "wake someone up." If you wouldn't want a page for it, don't use ERROR
  - INFO should be sparse enough so an engineer can follow the story without scrolling past noise.
  - DEBUG should be detailed enough that turning it on for one specific user gives you the full picture.

Metrics - The Four Standard Types

- All four are first-class types in Prometheus and most modern metrics systems
- Counter
  - Only goes up (or resets to zero); e.g., total requests served
  - Can compute a rate from it
    - *For example (rate(http_requests_total[5m]) gives requests per second*
- Gauge
  - Goes up and down; e.g., active connections, memory used at the current time
- Summary
  - Similar to histogram but computes quantiles client-side
  - Exists for historical reasons
  - In ew code, prefer histograms unless you have a specific reason for summaries.

Metrics - The Four Standard Types

- Histogram
  - Distribution of values; e.g., request latency at p50, p95, p99
  - Measures latency
  - A simple average of request times lies to you
  - A service that serves 99 requests in 10ms and 1 request in 30 seconds has a mean of about 300ms
  - This makes it look slow when actually one outlier is the entire problem

Metrics - Examples

- Counter:
  - http_requests_total{service="payment", status="500"}
- Counter
  - database_queries_total{table="orders", operation="select"}
- Gauge
  - jvm_memory_used_bytes{area="heap"}
- Gauge ;
  - active_websocket_connections
- Histogram:
  - http_request_duration_seconds{service="payment", endpoint="/checkout"}

The Four Golden Signals

- If you can only measure four things, measure these
  - Latency - how long do requests take to serve?
  - Traffic - how many requests are we handling?
  - Errors - how many requests are failing?
  - Saturation - how close to capacity is the system?
- Applies to any user-facing service regardless of what it does
- From Google's Site Reliability Engineering book
  - Has become the most widely-cited "what should I measure" framework in the industry.

RED and USE

- Two common frameworks
- RED method (for services):
  - Rate, Errors, Duration
  - Designed for microservices and request/response systems
  - Simpler than the four golden signals; collapses traffic and saturation
- USE method (for resources):
  - Utilization, Saturation, Errors
  - Designed for hardware and infrastructure (CPU, disk, network)
- Use RED for "is my service healthy?" - Use USE for "is my host healthy?"
- Both complement the golden signals; teams often pick one and run with it

Distributed Tracing

- A trace represents one request's full journey through a distributed system
- A span is one operation within that journey
  - E.g., a single service call, a database query
  - Spans have parent-child relationships, forming a tree
- The trace ID propagates from service to service in HTTP headers (traceparent)
- Every span records: start time, duration, service, operation, status, attributes

Distributed Tracing

- Directly addresses the "where did time go?" question in a microservice system
- The mental mode
  - Imagine the request as a tree where the root is the user's HTTP request
  - Each child is a downstream call to another service, to a database, to a cache, etc.
  - Each node in the tree is a span.
  - The tree structure is what lets you see, for instance
    - *That 90% of a request's time was spent in one specific database query four services deep.*
  - The key technical detail is context propagation
    - *When service A calls service B, it has to pass the trace ID along (in HTTP headers, message metadata, etc.)*
    - *So service B knows it's part of the same trace.*
    - *OpenTelemetry handles this automatically for most frameworks; you don't write the propagation code yourself in modern stacks.*

Distributed Tracing

- One picture; the bottleneck is obvious
- Tools: Jaeger, Zipkin, Tempo; all consume OpenTelemetry data

```
[POST /checkout]                                  420ms
├── [auth-service: validate-token]                 12ms
├── [cart-service: get-cart]                       45ms
│   └── [redis: GET cart:user:12345]                3ms
├── [inventory-service: check-stock]               38ms
│   └── [postgres: SELECT FROM inventory]          22ms
├── [pricing-service: compute-total]               28ms
└── [payment-service: charge]                     295ms  <-- slow
    └── [stripe-api: POST /charges]               287ms  <-- the cause
```

Health Checks

- Liveness and Readiness
- Services expose a health endpoint (e.g., /health or /actuator/health)
  - Liveness probe - "Is the process alive?" If no, restart it.
  - Readiness probe - "Can the process serve traffic right now?" If no, stop routing requests to it.
- Kubernetes, ECS, and load balancers use these to make routing decisions
- Spring Boot Actuator provides both out of the box

Alerting Basics

- An alert is a metric or log condition that triggers a notification (email, Slack, pager)
- Good alerts indicate user-facing impact; bad alerts indicate noise
- Alert on symptoms, not causes ("checkout is slow" not "CPU is high")
- Alert fatigue is real; if a pager fires 50 times a day, people stop reading it
- Every alert should have an associated runbook: what to do when this fires

Alerting Basics

- The single biggest alerting mistake is alerting on too many things.
  - If an alert for every metric you are worried about, the alert volume balloons
  - Within months the team is ignoring pages.
- The fix is to alert only on conditions that meet two criteria:
  - The user can tell something is wrong
  - A human needs to do something about it right now
    - *Usually the response is documented in a “run book”*
  - "CPU is at 85%" usually fails both criteria
    - *The user can't tell, and maybe nobody needs to act.*
  - "Checkout error rate exceeded 1% for five minutes" passes both
    - *Users are seeing errors, and someone needs to investigate.*
  - Every alert should link to a run book with current diagnosis steps

SLOs and Error Budgets

- SLI (Service Level Indicator)
  - A specific metric ("99.9% of requests succeed")
- SLO (Service Level Objective)
  - The target for an SLI over a time window
- Error budget
  - The inverse: 0.1% failure budget per month = ~43 minutes
- When the budget is healthy: ship features fast; when depleted: stop and fix reliability
- Originated at Google; now standard practice at most cloud-native companies

SLOs and Error Budgets

- SLOs turn reliability into an engineering decision rather than an emotional one
  - The framework gives you a number that says "we have spent X% of our allowed failure for this period."
  - When the budget is healthy, the team can take risks; deploy on Friday, run experiments, roll out new features.
  - When the budget is exhausted, the team shifts focus to reliability work until the budget recovers.
- This is what prevents two failure modes
  - Over-investing in reliability when users don't need it
  - Under-investing in reliability when users are suffering.

Dashboards

- A dashboard is a curated set of charts that answer a specific question
- One dashboard per audience:
  - Service health for the on-call engineer
  - Business metrics for the product manager
- Avoid wall-of-charts; if no one looks at a chart, delete it
- Standard tool: Grafana; open source, vendor-neutral, works with Prometheus, Loki, and others
- Most metric and log vendors provide their own dashboarding UIs as well

Dashboards

- Every chart on a dashboard exists because some person looks at it for some reason
  - If you can't name the person and the reason, delete the chart
- Different audiences need different dashboards
  - The on-call engineer wants golden-signals charts
  - The product manager wants conversion funnels
  - The CFO wants infrastructure cost
- One dashboard cannot serve all of them
- Trying to force all charts into on dashboard creates a useless dashboard for everyone.

Observability as a Quality Practice

- Tests catch bugs you anticipated
- Observability catches bugs you didn't
- Production telemetry should feed back into new test cases
- "Why didn't we catch this in testing?"; look at the trace, write the test
- The two are complementary; neither replaces the other

Observability as a Quality Practice

- Testing is what you do before deploy
- Observability is what you do after.
- Both are necessary
  - The feedback loop is the important part
  - When something breaks in production, the question after fixing it should always be "could a test have caught this, and if so, do we have that test now?"
  - Often the answer is yes; the production telemetry told you the exact condition that broke, and you can write a unit test or integration test that asserts the system handles it
  - Over time, this turns incidents into test cases, which means the same bug never bites you twice. Teams that do this religiously have remarkably stable systems.

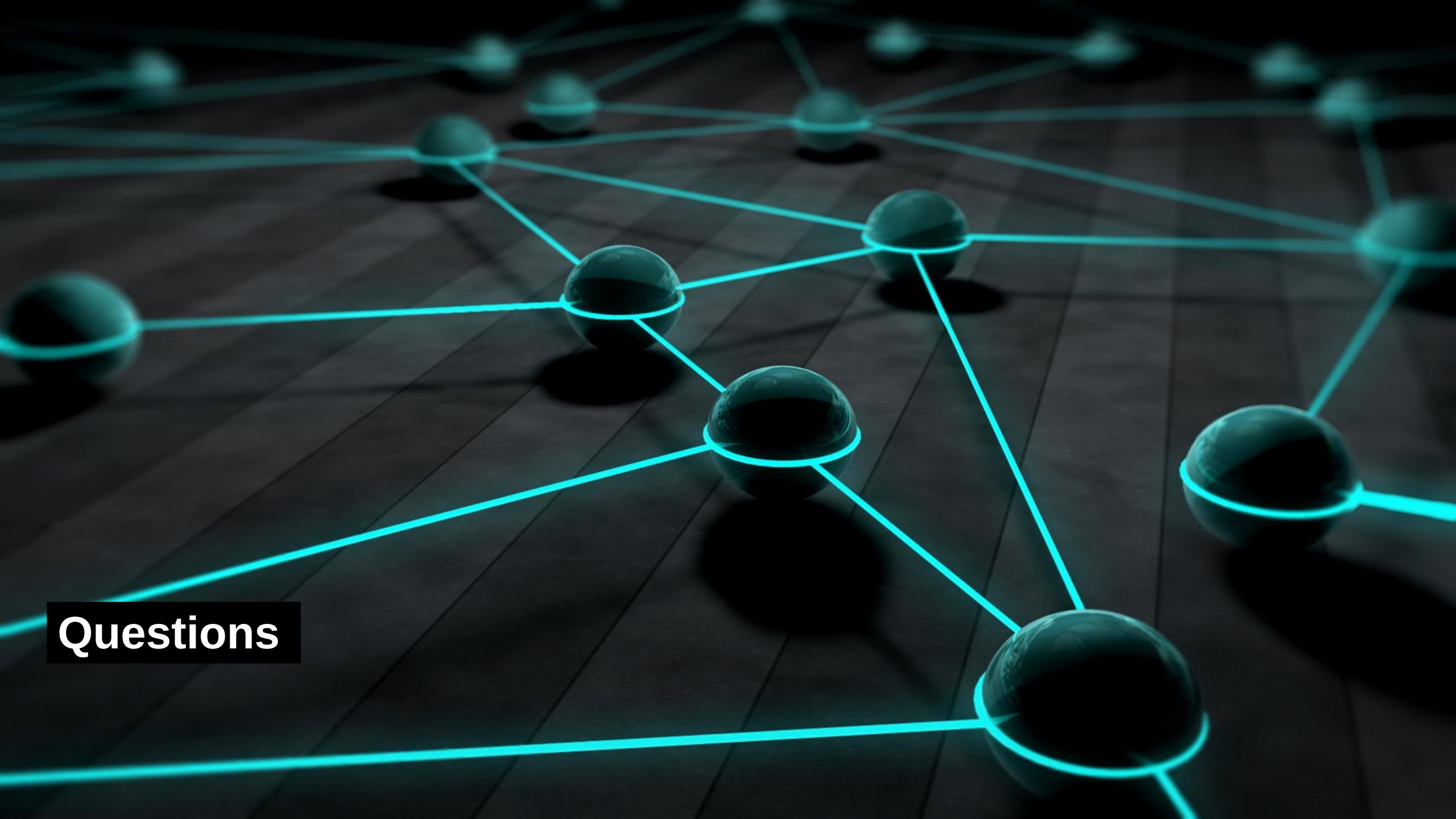
Observability in CI/CD

- Smoke tests after deploy exercise the observability endpoints
- Canary deployments use real production metrics to decide whether to proceed
- If error rate spikes during canary; roll back automatically
- "Deploy" is not the end of the cycle; observability is the feedback loop
- Mature pipelines treat metrics as deployment gates

The Tooling Landscape

- OpenTelemetry is the only entry in the "standards"
  - Because it's the vendor-neutral way to emit telemetry
  - Everything else either produces or consumes OpenTelemetry data
- The open-source stack (Prometheus + Grafana + Loki + Tempo) is what most cloud-native shops run when they want to avoid vendor lock
- Commercial vendors are polished and powerful but expensive
  - Typically $50-200 per host per month.
-

Pillar	Open source	Commercial
Standards	OpenTelemetry (instrumentation API)	-
Logs	ELK / EFK stack, Loki	Splunk, Datadog Logs
Metrics	Prometheus + Grafana	Datadog, New Relic
Traces	Jaeger, Zipkin, Tempo	Honeycomb, Datadog APM
All-in-one	SigNoz, OpenObserve	Datadog, New Relic, Dynatrace
Spring Boot help	Spring Boot Actuator (built-in)	-


Questions